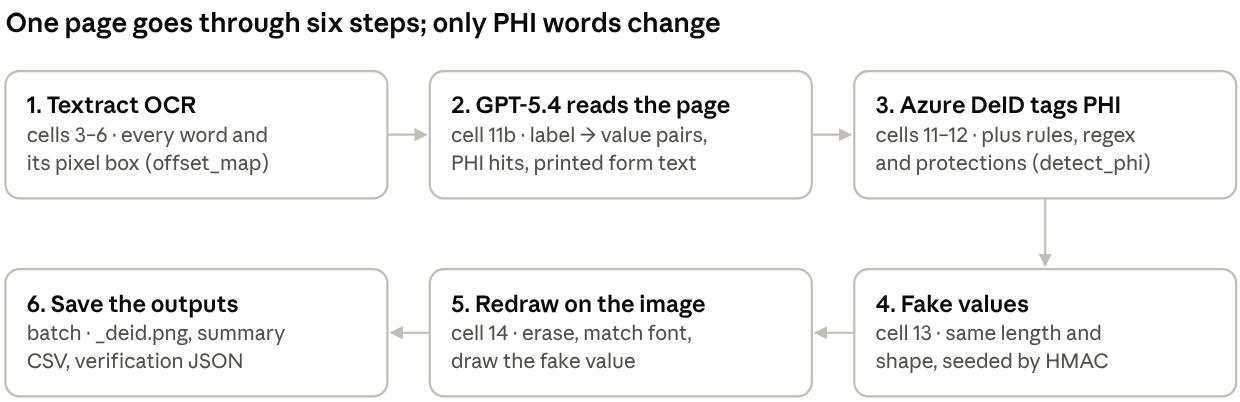
PHI De-identification Notebook — Line-by-Line Code Walkthrough

Oct 23, 2026 · @Anand

How to read this doc

This is the code-level companion to the *Complete Guide*. It goes through `final_v3_basic_ocr_vlm_23_oct.ipynb` cell by cell. For every cell you get:

1. **The code** (the real code from the notebook; very long cells are shown function by function).
2. **Line-by-line notes**: what each line or block does and why it is there.
3. **An example**: what goes in and what comes out, using the same sample page throughout.



notebook pipeline · 6 steps, cell numbers in each box

Each step only adds information to the same two objects: `full_text` (the page as one string) and `offset_map` (one entry per word with its position in the string and on the image). Steps 2 and 3 mark words as PHI, step 4 gives each marked word a `new_text`, step 5 draws it.

The sample page used in every example (a CMS-1500 claim form, made-up values):

Field on the form	Printed label	Value
Box 1a	`1a. INSURED'S I.D. NUMBER`	`123456789`
Box 2	`2. PATIENT'S NAME (Last Name, First Name, Middle Initial)`	`SMITH, CAROLINE M`
Box 3	`3. PATIENT'S BIRTH DATE` + captions `MM DD YY`	`03 15 1980`
Box 5	`5. PATIENT'S ADDRESS (No., Street)`	`482 MAPLE AVE`
Box 5	`CITY` / `STATE` / `ZIP CODE` / `TELEPHONE`	`SPRINGFIELD` / `IL` / `62704` / `(217) 555-0142`

The goal: every value in the right column changes; every label, caption (`MM DD YY`) and the form title stay exactly as printed.

Words used in this doc

- **Word id**: Textract's unique id for one word on the page. Everything in the notebook refers to words by this id.
- **Offset**: the position of a character in `full_text` (0 = first character). Azure answers in offsets, so offsets are how its answer is mapped back to words.
- **Flag**: a word marked as PHI, with its kind (name, date, id…) and which detector found it.
- **Surrogate**: the fake value that replaces a real one.

Cells 1, 1b and 2: libraries, debug printers, configuration

Cell 1 — Import libraries

```
import boto3        # AWS SDK for Python — Textract OCR
import json          # JSON parsing for API responses
import os            # File path operations
from collections import Counter  # Block type counting
from icp.authenticator.aws.boto3_services import boto3_clients as aws_client
import ssl           # Certificate handling
import sys           # Platform detection
import tempfile      # Temporary file creation
import urllib.error  # HTTP error handling
import urllib.request  # HTTP requests
```

Line	Why it is needed
`boto3`	Amazon's Python library. Used only to call Textract.
`aws_client`	Your company's wrapper around boto3. It signs in to AWS for you and, with `@aws_client.refresh_on_expire`, gets new credentials when the old ones expire in the middle of a batch.
`json`	Azure and GPT answer in JSON; this turns it into Python dicts.
`os`	Reads settings from environment variables (`os.environ`).
`Counter`	Counts Textract block types (how many LINE, WORD blocks) for the printout.
`ssl`, `sys`, `tempfile`	Used in cell 9 to build the company certificate file (Netskope) so HTTPS calls are trusted.
`urllib.request`, `urllib.error`	Calls the Azure DeID REST API directly, without an extra SDK.

Other libraries (`re`, `cv2`, `numpy`, `PIL`, `openai`, `hmac` …) are imported in the cells that use them.

Cell 1b — Debug printers

```
DEBUG = True            # single-image run (sections 6, 11b, 12, 13, 14)
DEBUG_BATCH = False     # every page of the batch run (very long output)
DEBUG_MAX_ROWS = 120    # rows printed per table
```

- `DEBUG` : when True, each later cell prints what it produced for the test image.
- `DEBUG_BATCH` : same printouts, for every page of the batch. Off by default because 100 pages × 5 tables is a lot of output.
- `DEBUG_MAX_ROWS` : stops a table after 120 rows and prints "… N more rows".

Two helpers do the formatting:

```
def _dbg_title(t):
    print("\n" + "=" * 100 + f"\n{t}\n" + "=" * 100)

def _dbg_rows(rows, header):
    widths = [max(len(str(h)), *(len(str(r[i])) for r in rows)) if rows else
len(str(h)) for i, h in enumerate(header)]
    widths = [min(w, 60) for w in widths]
    fmt = "  ".join("{:<%d}" % w for w in widths)
    ...
```

- `_dbg_title` prints a banner line of `=` above and below the stage name.
- `_dbg_rows` works out the widest value in each column (capped at 60 characters), builds a format string like `{:<4} {:<8} {:<60}`, and prints the header, a line of dashes, then the rows. This is a plain-text table without any extra library.

The five printers, one per stage:

Function	Called in	Prints
`debug_ocr(offset_map)`	Cell 6	Each OCR line, its lowest word confidence, words under 70% confidence, handwritten words
`debug_vlm(stats, kv_pairs, vlm_hits, protected_ids, word_by_id)`	Cell 11b	Every field GPT returned and whether it was found among the OCR words; free-text PHI; the protected words

Function	Called in	Prints
`debug_tagging(azure_entities, flags, offset_map, blocked)`	Cell 12	3a Azure's entities, 3b every flagged word and which detectors found it, 3c attempts to change protected words that were stopped
`debug_surrogates(entities)`	Cell 13	original → fake for every PHI value
`debug_render(actions, word_by_id)`	Cell 14	what happened to each PHI word on the image

A detail in `debug_tagging`, line by line:

```
part = "".join(ch if m else "·" for ch, m in zip(w['text'], fl["mask"]))
```

The `mask` says which characters of a word are PHI. For the OCR word `DOB:03/15/1980` the mask is False for `DOB:` and True for the date, so the column shows `····03/15/1980`. You can see at a glance that the label part will not be changed.

Example (Stage 1 for the sample page):

```
================================================================================
========================
STAGE 1 - OCR (Textract DetectDocumentText): lines as read
================================================================================
========================
line  min_conf  text                                  low_conf_words
handwriting
----  --------  ------------------------------------  --------------  ------
-----
0     99        HEALTH INSURANCE CLAIM FORM
4     98        1a. INSURED'S I.D. NUMBER
5     99        123456789
6     97        2. PATIENT'S NAME (Last Name, First Name, Middle Initial)
7     96        SMITH, CAROLINE M
```

The printouts contain real PHI. Clear the notebook outputs before sharing it.

Cell 2 — Azure DeID configuration

```
from dotenv import load_dotenv
load_dotenv()  # reads the .env file in the project root

ENDPOINT = os.environ["AZURE_DEID_ENDPOINT"]
TENANT_ID = os.environ["AZURE_DEID_TENANT_ID"]
SCOPE = os.environ.get("AZURE_DEID_SCOPE", "https://deid.azure.com/.default")
API_VERSION = os.environ.get("AZURE_DEID_API_VERSION", "2024-11-15")
```

Line	What it does
`load_dotenv()`	Reads `.env` and puts each `KEY=value` into the environment, so no secret is written in the notebook.
`os.environ["AZURE_DEID_ENDPOINT"]`	Square brackets: the setting is required. If it is missing the cell stops with a `KeyError` at once, instead of failing later with a confusing network error.
`os.environ.get(..., default)`	Optional settings with a sensible default.
`SCOPE`	What the sign-in token is for: the Azure DeID service.
`API_VERSION`	The DeID REST API version. Pinned so a new Azure version cannot change the answer format without you knowing.

The same `.env` also holds the GPT-5.4 settings (cell 11b) and `DEID_SURROGATE_KEY` (cell 13).

Cells 3–5: loading the image and running Textract OCR

Cell 3 — `load_image`

```
SUPPORTED_IMAGE_FORMATS = ['.png', '.jpg', '.jpeg', '.bmp', '.gif']

def load_image(image_path):
    if not os.path.exists(image_path):
        raise FileNotFoundError(f"Image file not found: {image_path}")
    file_ext = os.path.splitext(image_path)[1].lower()
    if file_ext not in SUPPORTED_IMAGE_FORMATS:
        raise ValueError(f"Unsupported format: {file_ext}. ...")
    print(f"✓ Image loaded: {image_path}")
    return image_path

IMAGE_PATH = r"C:\Users\RSahu\...\CMS 1500 202604230008171_page_1.png"
IMAGE_PATH = load_image(IMAGE_PATH)
```

Line	What it does
`os.path.exists`	Stops early with a clear message if the path is wrong.
`os.path.splitext(...)[1].lower()`	Takes the extension (`.PNG` → `.png`) so the check is not case-sensitive.
`raise ValueError`	Refuses PDFs and multi-page TIFFs: Textract's synchronous call reads one image. Split PDFs into page images first.
`r"C:\..."`	The `r` means a raw string, so the backslashes in the Windows path are not read as escape codes.

This cell is for the single test image. The batch cells at the end find the images themselves.

Cell 4 — Textract basic OCR

```
with open(IMAGE_PATH, "rb") as f:
    image_bytes = f.read()

print(f"Image size: {len(image_bytes)/1024:.1f} KB")
assert len(image_bytes) < 10 * 1024 * 1024, "Image exceeds 10MB sync limit"

@aws_client.refresh_on_expire
def get_ocr(image_bytes):
    textract = aws_client.get("textract")
    return textract.detect_document_text(Document={'Bytes': image_bytes})

response = get_ocr(image_bytes)
blocks = response['Blocks']
type_counts = Counter(b['BlockType'] for b in blocks)
```

Line	What it does
`open(..., "rb")`	Reads the file as raw bytes ("rb" = read binary). Textract takes the bytes, not a path.
`assert ... < 10 MB`	Textract refuses bytes over 10 MB in a direct call. Checking first gives a clear message instead of an AWS error.
`@aws_client.refresh_on_expire`	If the AWS credentials expired, the wrapper gets new ones and repeats the call. Matters in long batches.
`aws_client.get("textract")`	A ready, signed-in Textract client.
`detect_document_text`	Basic OCR: lines and words with their boxes and confidence. No form pairing, no tables. About $0.0015 per page. GPT-5.4 gives the label → value context instead (cell 11b).
`response['Blocks']`	Everything Textract found, as a flat list of blocks.
`Counter(...)`	Counts block types for the printout.

Example output:

```
Image size: 412.6 KB
Total blocks returned: 389
Block type counts: {'PAGE': 1, 'LINE': 74, 'WORD': 314}
```

What a block looks like (the word SMITH, — Textract keeps the comma attached to the word; shortened):

```
{
  "BlockType": "WORD",
  "Id": "7c2f...e91",
  "Text": "SMITH,",
  "Confidence": 99.4,
  "TextType": "PRINTED",
  "Geometry": {
    "BoundingBox": {"Left": 0.0431, "Top": 0.1604, "Width": 0.0548, "Height":
0.0119},
    "Polygon": [{"X": 0.0431, "Y": 0.1604}, {"X": 0.0979, "Y": 0.1605},
                {"X": 0.0979, "Y": 0.1723}, {"X": 0.0431, "Y": 0.1722}]
  }
}
```

Positions are **fractions of the page** (0.0431 = 4.31% from the left edge), not pixels. Cell 6 turns them into pixels. A LINE block has the same fields plus `Relationships` → `CHILD` → the ids of its WORD blocks; that link is how words are grouped into lines.

Cell 5 — Print the LINE blocks

```
lines = [b for b in blocks if b['BlockType'] == 'LINE']
for line in lines:
    bbox = line['Geometry']['BoundingBox']
    entry = (f"Type: {line['BlockType']} | Confidence:
{line['Confidence']:.2f}% | "
             f"Location(Left={bbox['Left']:.4f}, Top={bbox['Top']:.4f}, ...)
| Text: {line['Text']}")
    print(entry)
    output_lines.append(entry)

with open(output_txt_path, "w", encoding="utf-8") as f:
    f.write("\n".join(output_lines))
```

Only for checking: it prints each line with its confidence and position and saves the same text to `extracted_text_detailed.txt`. Nothing later uses this file. Example line:

```
Type: LINE | Confidence: 99.12% | Location(Left=0.0431, Top=0.1604,
Width=0.1693, Height=0.0121) | Text: SMITH, CAROLINE M
```

The next cell (`lines`) just shows the same list in Jupyter.

Cell 6: `build_word_index` — the word index everything else uses

This is the most important small function in the notebook. Azure and the rules work on **text**; the image work needs **pixels**. `build_word_index` builds both at once and links them.

```
def build_word_index(blocks, img_w, img_h):
    block_map = {b['Id']: b for b in blocks}                      # (1)
    line_blocks = [b for b in blocks if b['BlockType'] == 'LINE'] # (2)

    full_text = ""
    offset_map = []

    for line_idx, line in enumerate(line_blocks):                 # (3)
        word_ids = []
        for rel in line.get('Relationships', []):
            if rel['Type'] == 'CHILD':
                word_ids.extend(rel['Ids'])                       # (4)

        for word_id in word_ids:
            word_block = block_map.get(word_id)
            if not word_block or word_block['BlockType'] != 'WORD':
                continue                                          # (5)

            word_text = word_block['Text']
            bbox = word_block['Geometry']['BoundingBox']
            poly = word_block['Geometry'].get('Polygon') or []

            angle = 0.0                                           # (6)
            if len(poly) >= 2:
                dx = (poly[1]['X'] - poly[0]['X']) * img_w
                dy = (poly[1]['Y'] - poly[0]['Y']) * img_h
                if abs(dx) + abs(dy) > 0:
                    angle = math.degrees(math.atan2(dy, dx))

            start = len(full_text)                                # (7)
            full_text += word_text
            end = len(full_text)
```

```
            offset_map.append({                                      # (8)
                "idx": len(offset_map),
                "start": start, "end": end, "text": word_text,
                "bbox": bbox, "confidence": word_block['Confidence'],
                "word_id": word_id, "line_idx": line_idx,
                "px": (int(math.floor(bbox['Left'] * img_w)),      # (9)
                       int(math.floor(bbox['Top'] * img_h)),
                       int(math.ceil((bbox['Left'] + bbox['Width']) *
img_w)),
                       int(math.ceil((bbox['Top'] + bbox['Height']) *
img_h))),
                "poly": [(pt['X'] * img_w, pt['Y'] * img_h) for pt in poly]
if len(poly) >= 3 else None,
                "angle": angle,
                "handwriting": word_block.get('TextType') == 'HANDWRITING',
# (10)
            })
            full_text += " "    # space between words           # (11)
        full_text += "\n"       # newline between lines

    return full_text, offset_map
```

Line by line

1. `block_map`: a dictionary id → block, so a word can be found by its id at once instead of searching the whole list each time.
2. `line_blocks`: only the LINE blocks, in the order Textract read them (top to bottom, left to right). This order becomes the reading order of `full_text`.
3. `enumerate` gives each line a number, `line_idx`. Words with the same `line_idx` were printed on one line; cell 14 uses this to pick one font per line.
4. A LINE's `CHILD` relationship lists its WORD ids, in reading order.
5. Skips anything that is not a word (Textract can list a checkbox, `SELECTION_ELEMENT`, as a child).
6. **Tilt of the word.** `poly[0]` is the top-left corner and `poly[1]` the top-right corner. The difference across (`dx`) and down (`dy`) is converted to **pixels first** (a page is taller than it is wide, so fractions would give a wrong angle). `atan2(dy, dx)` is the angle of the top edge; `degrees` turns it into degrees. 0 = straight; 1.5 = the word slopes down 1.5° to the right. Cell 14 rotates the fake text by this angle so it sits on the same slope.

7. `start` / `end`: where this word sits in `full_text`. `full_text[start:end]` is always exactly the word.
8. One entry per word in `offset_map`. `idx` is its position in the list (reading order).
9. **Pixel box.** Fraction × image size = pixels. `floor` for the left and top, `ceil` for the right and bottom, so rounding never cuts a sliver off the letters; `int` because image slicing needs whole numbers.
10. `handwriting`: Textract marks handwritten words. Shown in the debug printout; useful when checking why a value was missed.
11. A space after every word and a newline after every line, so `full_text` reads like the page and Azure sees normal text.

After the function:

```
with Image.open(IMAGE_PATH) as _im:
    IMG_W, IMG_H = _im.size
full_text, offset_map = build_word_index(blocks, IMG_W, IMG_H)
word_by_id = {e['word_id']: e for e in offset_map}
if DEBUG:
    debug_ocr(offset_map)
```

`Image.open(...).size` reads the width and height without loading the whole picture. `word_by_id` is the same list keyed by word id, for quick look-ups in later cells.

Worked example (page 2550 × 3300 px, i.e. a letter-size page scanned at 300 dpi)

Input: the `SMITH,` block from cell 4 (`Left 0.0431, Top 0.1604, Width 0.0548, Height 0.0119`).

Value	Calculation	Result
x0	floor(0.0431 × 2550) = floor(109.9)	109
y0	floor(0.1604 × 3300) = floor(529.3)	529
x1	ceil(0.0979 × 2550) = ceil(249.6)	250
y1	ceil(0.1723 × 3300) = ceil(568.6)	569
dx	(0.0979 − 0.0431) × 2550	139.7 px
dy	(0.1605 − 0.1604) × 3300	0.33 px
angle	degrees(atan2(0.33, 139.7))	0.14° (practically straight)

If the text before this line is 212 characters long, the line `SMITH, CAROLINE M` adds:

```
full_text[212:218] = "SMITH,"     full_text[219:227] = "CAROLINE"
full_text[228:229] = "M"
```

and the entry for `SMITH,` is:

```
{"idx": 9, "start": 212, "end": 218, "text": "SMITH,", "confidence": 99.4,
 "word_id": "7c2f...e91", "line_idx": 7, "px": (109, 529, 250, 569),
 "poly": [(109.9, 529.3), (249.6, 529.7), (249.6, 568.6), (109.9, 568.3)],
 "angle": 0.14, "handwriting": False, "bbox": {...}}
```

Why this matters later: Azure says "a patient name at offsets 212–217". The notebook finds the word whose `start` – `end` covers 212–217 → `SMITH,` → its pixel box (109, 529, 250, 569) is the area to repaint. Only the characters 212–217 are PHI, so the comma is kept.

Cells 8–11: the label: value view, certificates, Azure sign-in and the DeID call

Cell 8 — `build_context_view` (function only)

```
def build_context_view(kv_pairs, table_cells, word_by_id):
    ctx_text, ctx_map = "", []

    def add(label, word_ids):
        nonlocal ctx_text
```

```
        ids = [i for i in word_ids if i in word_by_id]
        if not ids:
            return
        ctx_text += (label.strip() or "Value") + ": "
        for wid in ids:
            start = len(ctx_text)
            ctx_text += word_by_id[wid]['text']
            ctx_map.append({"start": start, "end": len(ctx_text), "word_id":
wid})
            ctx_text += " "
        ctx_text += "\n"

    for p in kv_pairs:
        add(p['key_text'], p['value_word_ids'])
    for c in table_cells:
        add(c['header_text'], c['word_ids'])
    return ctx_text, ctx_map
```

Why it exists. On a CMS-1500 the value is often far from its label, so in `full_text` Azure may see `123456789` on a line by itself and not know it is a member ID. This function writes a second text where every value sits right after its label: `1a. INSURED'S I.D. NUMBER: 123456789`.

Line	What it does
`def add(...)` inside the function	A small helper that appends one `label: value` line. `nonlocal ctx_text` lets it change the outer variable.
`ids = [i for i in word_ids if i in word_by_id]`	Keeps only word ids that really exist in the OCR index.
`label.strip() or "Value"`	If a label is empty, writes `Value:` so the line still has the same shape.
`ctx_map.append(...)`	Only the **value** words are recorded with their offsets in `ctx_text`. The label part has no entry, so if Azure tags label words, nothing on the image changes.
`for p in kv_pairs`	The pairs come from GPT-5.4 (cell 11b).

Line	What it does
`for c in table_cells`	Kept for compatibility; with basic OCR this list is empty.

Example, built from two GPT pairs:

```
ctx_text =
"1a. INSURED'S I.D. NUMBER: 123456789 \n2. PATIENT'S NAME (Last Name, First
Name, Middle Initial): SMITH, CAROLINE M \n"

ctx_map (first entries) =
[{"start": 27, "end": 36, "word_id": "a41..."},   # 123456789
 {"start": 98, "end": 104, "word_id": "7c2f...e91"}, # SMITH,
 ...]
```

If Azure tags offsets 98–103 of `ctx_text` as a name, `ctx_map` points to word `7c2f...e91`, which is the same `SMITH,` word as in cell 6, with the same pixel box.

Cell 9 — Company certificates (Netskope)

The company network (Netskope) inspects HTTPS traffic and signs it with its own certificate. Python does not trust that certificate by default, so calls to AWS, Azure and OpenAI would fail with `CERTIFICATE_VERIFY_FAILED`.

```
def export_netskope_ca():
    base_bundle = certifi.where()                        # Python's standard
list of trusted roots
    ...
    if sys.platform != "win32":
        return base_bundle if bundle_text else None     # not Windows:
nothing to add
    pems = []
    for store in ("ROOT", "CA"):
        for der, encoding, _ in ssl.enum_certificates(store):   # Windows
certificate store
            if encoding != "x509_asn":
                continue
            pem = ssl.DER_cert_to_PEM_cert(der)
            if b"netskope" in der.lower() or b"goskope" in der.lower():
                pems.append(pem)
    ...
    path = os.path.join(tempfile.gettempdir(), "NetskopeRootPlusCertifi.pem")
    # writes certifi's list + the Netskope certificates into that one file
    return path
```

Step	What it does
`certifi.where()`	Path to the normal list of trusted public certificates.
`ssl.enum_certificates(store)`	Reads the Windows certificate store, where IT installed the Netskope root.
`DER_cert_to_PEM_cert`	Converts each certificate from the binary Windows format to the text format Python reads.
`b"netskope" in der.lower()`	Keeps only the Netskope ones.
writes a temp `.pem`	One file = the public list + Netskope.

`configure_ca_bundle` then sets `REQUESTS_CA_BUNDLE`, `SSL_CERT_FILE` and `AWS_CA_BUNDLE` to that file, so `boto3`, `azure-identity`, `openai` and `urllib` all trust it. Off the company network, nothing is found and the defaults are used.

Cell 10 — Azure sign-in

```
credential = InteractiveBrowserCredential(tenant_id=TENANT_ID, timeout=240)
token = credential.get_token(SCOPE)
```

- Opens a browser window; you sign in with an Azure account that has the **DeID Data Owner** role. You get 240 seconds.
- `get_token(SCOPE)` asks for a token for the DeID service. `credential` keeps it in memory and renews it when it expires, so later calls do not ask you again.
- If sign-in fails, `raise` stops the notebook here, rather than at the first Azure call.

Cell 11 — The Azure DeID call

```
AZURE_MAX_CHARS = 20000

def call_azure_deidentification(text, operation="Tag"):
    token = credential.get_token(SCOPE)                                  # (1)
    body = json.dumps({"inputText": text, "operation": operation}).encode()
# (2)
    request = urllib.request.Request(
        f"{ENDPOINT}/deid?api-version={API_VERSION}",                     # (3)
        data=body, method="POST",
        headers={"Content-Type": "application/json",
                 "Authorization": f"Bearer {token.token}"})
    try:
        with urllib.request.urlopen(request, timeout=60) as response:
            return json.loads(response.read())                           # (4)
    except urllib.error.HTTPError as exc:
        print(f"❌ Azure DeID error: {exc.code} {exc.reason}")
        print(exc.read().decode(errors="replace")[:500])                 # (5)
        raise
```

1. A token for every call. `azure-identity` returns the cached one while it is valid, so this is cheap, and it avoids a 401 error once the first token expires.
2. The request body. `operation` is `"Tag"`: Azure only tells us **where** the PHI is and what kind it is; it does not change the text. (The old code sent `operationType`, which Azure ignored.)
3. The DeID endpoint from `.env` with the pinned API version.
4. The answer, as a Python dict.

5. On an HTTP error, prints the code and the first 500 characters of Azure's explanation, then stops.

Chunking — `_split_chunks`

```
def _split_chunks(text, limit):
    chunks, pos = [], 0
    while pos < len(text):
        end = min(len(text), pos + limit)
        if end < len(text):
            nl = text.rfind("\n", pos, end)
            if nl > pos:
                end = nl + 1
        chunks.append((pos, text[pos:end]))
        pos = end
    return chunks
```

Azure accepts a limited amount of text per request (about 50 KB). The text is cut into pieces of at most 20,000 characters, and each cut is moved back to the last line break (`rfind("\n")`) so a name or date is never split across two requests. Each piece is returned with `pos`, its starting offset (its **base**), so Azure's offsets can be moved back into the whole text.

Text length	Chunks (base, length)	Azure calls
2,140 (one CMS-1500 page)	(0, 2140)	1
45,000 (a long typed page)	(0, 19,987), (19,987, 19,995), (39,982, 5,018)	3

A typical claim page is one chunk. Azure bills by the amount of text, not by the number of calls, so chunking does not add cost.

`_as_int`: Azure gives each offset in three encodings, e.g. `{"utf8": 98, "utf16": 98, "codePoint": 98}`. Python counts characters as code points, so `codePoint` is used. The numbers only differ when the text has special characters (é, emoji).

`azure_tag(text)`, step by step:

```
for base, chunk in _split_chunks(text, AZURE_MAX_CHARS):
    result = call_azure_deidentification(chunk, "Tag")
    tagger = result.get("taggerResult") or result.get("tagger_result")
    for e in tagger.get("entities", []):
        s, n = _as_int(e["offset"]), _as_int(e["length"])
        etext = e.get("text")
        if etext and chunk[s:s + n] != etext:        # safety check
            found = chunk.find(etext, max(0, s - 100))
            ...
        entities.append({"start": base + s, "end": base + s + n,
                         "category": e.get("category", "Unknown"),
                         "confidence": float(e.get("confidenceScore", ...) or
1.0),
                         "text": text[base + s: base + s + n]})
```

- `base + s`: the offset inside the chunk plus where the chunk starts = the offset in the whole text.
- **Safety check**: if the text at Azure's offset is not the entity text Azure reports, the code searches for it nearby (from 100 characters before). Counts of moved and lost entities are printed. This protects against offset mistakes, which would otherwise repaint the wrong word.

Example Azure answer and the result:

```
{"taggerResult": {"entities": [
  {"category": "Patient", "offset": {"codePoint": 212}, "length":
{"codePoint": 5},
   "text": "SMITH", "confidenceScore": 0.97},
  {"category": "Date", "offset": {"codePoint": 318}, "length": {"codePoint":
10},
   "text": "03 15 1980", "confidenceScore": 0.93}
]}}
```

```
[{"start": 212, "end": 217, "category": "Patient", "confidence": 0.97,
"text": "SMITH"},
 {"start": 318, "end": 328, "category": "Date", "confidence": 0.93, "text":
"03 15 1980"}]
```

`azure_tag` is called twice per page in cell 12: on `full_text` and on `ctx_text`.

Cell 11b: GPT-5.4 reads the page

Basic OCR knows **where** every word is but not **what** it is. GPT-5.4 looks at the image and says "this is the patient's name, this is the birth date, this is printed form text". The cell then finds GPT's text among the OCR words, so GPT never decides a position; Textract does.

The cell has five parts: settings, the GPT call, text matching, form-text protection, and the run for this page.

Part 1 — Settings and the client

```
USE_VLM = True
VLM_MAX_WIDTH = 2000          # image is resized to at most this width before
sending
VLM_MAX_TOKENS = 16000        # GPT-5.x counts reasoning tokens here too
VLM_MATCH_THRESHOLD = 0.85    # how similar a VLM value must be to OCR words
VLM_CACHE_DIR = None          # folder to reuse answers on re-runs (contains
PHI)

AZURE_OPENAI_ENDPOINT = os.getenv("endpoint") or
os.getenv("AZURE_OPENAI_ENDPOINT")
AZURE_OPENAI_DEPLOYMENT = os.getenv("deployment") or
os.getenv("AZURE_OPENAI_DEPLOYMENT")
AZURE_OPENAI_API_KEY = os.getenv("subscription_key") or
os.getenv("AZURE_OPENAI_API_KEY")
AZURE_OPENAI_API_VERSION = os.getenv("api_version") or ... or "2024-10-21"

_missing = [...]
if USE_VLM and _missing:
    raise ValueError(f"Missing Azure OpenAI settings in .env: ...")

vlm_client = AzureOpenAI(api_version=..., azure_endpoint=..., api_key=...) if
USE_VLM else None
```

- Each setting is read under your `.env` names (`endpoint`, `deployment`, `subscription_key`) and, as a second choice, under the standard Azure names.
- `_missing` lists any required setting that is empty and stops with a clear message, instead of a confusing 401 later.
- `VLM_MAX_TOKENS = 16000`: GPT-5.x "thinks" before answering and those thinking tokens count too. A busy UB-04 needs a long answer; too low a value cuts the JSON off.

Part 2 — The prompt and the call

The prompt (`VLM_PROMPT`) asks for JSON in exactly this shape:

```
{"fields": [{"label": "...", "value": "...", "category": "...", "is_phi":
true}],
 "free_text_phi": [{"value": "...", "category": "..."}],
 "printed_text": ["..."]}
```

Part of the prompt	Why it is written that way
label "exactly as printed, including its box number"	So the label can be found among the OCR words and protected.
value "copied EXACTLY … Do not correct, reformat or complete"	The value is searched for in the OCR text. If GPT wrote `03/15/1980` for a printed `03 15 1980` , it might not match.
one entry per address part	Street, city, state and zip are different PHI kinds and get different fake values.
fixed list of categories	Each category maps to one kind used by cells 12 and 13 (`VLM_KIND`).
`is_phi` false for codes, charges, sex, yes/no	Diagnosis codes and amounts are not identifiers and must stay.
`free_text_phi`	Identifiers outside labelled boxes: stamps, signatures, notes.
`printed_text`	Every pre-printed word of the blank form, so the notebook knows what must never change.
"Do not invent values"	GPT must not fill in empty boxes.

Category → kind:

```
VLM_KIND = {"name": "name", "date": "date", "phone": "phone", "fax": "phone",
"email": "email",
            "street": "street", "city": "city", "state": "state", "zip":
"zip", "country": "country",
            "organization": "org", "hospital": "org", "profession":
"profession"}

def vlm_kind(category, value=""):
    c = str(category or "").strip().lower()
    if c == "age_over_89":
        digits = re.sub(r"\D", "", str(value))
        return "id" if digits and int(digits) > 89 else None
    return VLM_KIND.get(c, "id")
```

Anything not in the table (ssn, member ID, account number…) becomes kind `id`. An age is PHI only above 89 (HIPAA Safe Harbor), so `age_over_89` with value `72` returns `None` and is ignored.

```
def _image_b64(image_path, max_width=VLM_MAX_WIDTH):
    image = Image.open(image_path).convert("RGB")
    if image.width > max_width:
        scale = max_width / image.width
        image = image.resize((int(image.width * scale), int(image.height *
scale)), Image.Resampling.LANCZOS)
    buffer = io.BytesIO()
    image.save(buffer, format="PNG")
    return base64.b64encode(buffer.getvalue()).decode("utf-8")
```

A 2550 px page is shrunk to 2000 px wide (same proportions, `LANCZOS` keeps small print sharp), saved as PNG in memory and turned into base64 text, which is how an image is put into a chat message.

```
def vlm_extract(image_path):
    cache = Path(VLM_CACHE_DIR) / f"{Path(image_path).stem}_vlm.json" if
VLM_CACHE_DIR else None
    if cache and cache.exists():
        return json.loads(cache.read_text(encoding="utf-8"))           # (1)
    response = vlm_client.chat.completions.create(
        model=AZURE_OPENAI_DEPLOYMENT,
        messages=[
            {"role": "system", "content": "You read healthcare document
images and return strict JSON only."},
```

```
            {"role": "user", "content": [
                {"type": "text", "text": VLM_PROMPT},
                {"type": "image_url", "image_url": {"url":
f"{...}", "detail": "high"}},
            ]},
        ],
        max_completion_tokens=VLM_MAX_TOKENS)                                  #
(2)
    choice = response.choices[0]
    if choice.finish_reason == "length":
        raise RuntimeError("VLM output cut off ...")
# (3)
    result = _parse_json(choice.message.content)
# (4)
    result.setdefault("fields", []); result.setdefault("free_text_phi", []);
result.setdefault("printed_text", [])
    if cache: ...write result...
# (5)
    return result
```

1. With a cache folder set, a page already sent once is read from disk: no cost, same answer.
2. One chat call: a system message (role), then the prompt and the image. `detail: "high"` makes GPT look at the image at full resolution, needed for small form print. On Azure, `model=` is the **deployment** name.
3. `finish_reason == "length"` means the answer was cut off; a half JSON would lose fields, so it stops with a clear message.
4. `_parse_json` removes a ```` ```json ```` fence if GPT added one and, if there is extra text around the JSON, takes the part from the first `{` to the last `}`.
5. `setdefault` makes sure all three lists exist even if GPT left one out.

Example answer for the sample page (shortened):

```
{"fields": [
  {"label": "1a. INSURED'S I.D. NUMBER", "value": "123456789", "category":
"health_plan_id", "is_phi": true},
  {"label": "2. PATIENT'S NAME (Last Name, First Name, Middle Initial)",
"value": "SMITH, CAROLINE M", "category": "name", "is_phi": true},
  {"label": "3. PATIENT'S BIRTH DATE", "value": "03 15 1980", "category":
"date", "is_phi": true},
  {"label": "SEX", "value": "F", "category": "other", "is_phi": false},
  {"label": "CITY", "value": "SPRINGFIELD", "category": "city", "is_phi":
true}
 ],
 "free_text_phi": [{"value": "C. SMITH", "category": "name"}],
 "printed_text": ["HEALTH INSURANCE CLAIM FORM", "1a. INSURED'S I.D. NUMBER",
"MM DD YY", "MEDICARE MEDICAID TRICARE CHAMPVA"]}
```


Part 3 — Finding GPT's text among the OCR words

```
def _compact(s):
    return re.sub(r"[^a-z0-9]", "", str(s or "").lower())

def _similar(a, b):
    if not a or not b:
        return 0.0
    if min(len(a), len(b)) <= 3:
        return 1.0 if a == b else 0.0
    return SequenceMatcher(None, a, b).ratio()
```

- `_compact` lower-cases and removes everything except letters and digits: `"SMITH, CAROLINE M"` → `"smithcarolinem"`, `"03/15/1980"` → `"03151980"`. Spacing and punctuation differences between GPT and OCR no longer matter.
- `_similar` gives 0–1. Short values (≤ 3 characters, like `IL` or `F`) must match exactly, otherwise any two-letter word would count. Longer values use `SequenceMatcher.ratio()` = 2 × matching characters ÷ total characters.

```
CAPTION_TOKENS = {"MM", "DD", "YY", "YYYY", "CCYY"}

def _ocr_lines(offset_map):
    by_line = {}
    for w in offset_map:
        if w['text'].upper().strip(".,:;()|[]") in CAPTION_TOKENS:
            continue
        by_line.setdefault(w['line_idx'], []).append(w)
    return [sorted(ws, key=lambda w: w['idx']) for _, ws in
sorted(by_line.items())]
```

Groups OCR words by line, **leaving out the date captions**. On a CMS-1500 the OCR often reads the birth date line as `MM 03 DD 15 YY 1980`. Without the captions it is `03 15 1980`, which matches GPT's value.

```
def match_text(text_value, offset_map, lines=None):
    target = _compact(text_value)
    n = max(1, len(str(text_value).split()))                    # words in
GPT's value
    hits = []
    for line in (lines or _ocr_lines(offset_map)):
        for i in range(len(line)):
            for size in range(max(1, n - 1), n + 2):                # windows
of n-1, n, n+1 words
                win = line[i:i + size]
                if len(win) < size:
                    break
                wc = _compact(" ".join(w['text'] for w in win))
                score = _similar(target, wc)
                if score < VLM_MATCH_THRESHOLD and size == 1 and len(target)
>= 4 and target in wc:
                    score = VLM_MATCH_THRESHOLD                     # value
glued to other text
                if score >= VLM_MATCH_THRESHOLD:
                    hits.append((score, -len(win), win))
    chosen, used = [], set()
    for score, _neg, win in sorted(hits, key=lambda h: (-h[0], h[1])):
        ids = {w['word_id'] for w in win}
        if ids & used:
```

```
            continue
        used |= ids
        chosen.append((score, win))
    return chosen
```

It slides a window along every OCR line. The window has the same number of words as GPT's value, or one fewer or one more (OCR may split `555-0142` into two words or join two words). Each window is compared after `_compact`; a score of 0.85 or more is a hit. A value stuck to its label in one OCR word (`DOB:03/15/1980`) is accepted when GPT's value is contained in it. Hits are then sorted best first, and a word is used by one hit only.

Examples:

GPT value	OCR words	Compared	Score	Result
`03 15 1980`	`MM 03 DD 15 YY 1980` (captions skipped)	`03151980` vs `03151980`	1.00	match
`SMITH, CAROLINE M`	`SMITH,` `CAROLINE` `M`	`smithcarolinem` vs same	1.00	match
`CAROLINE M`	`CAROL1NE` `M` (OCR read l as 1)	`carolinem` vs `carol1nem`	0.89	match
`IL`	`IL`	short: exact	1.00	match
`IL`	`ILLINOIS`	short: not equal	0.00	no match
`03/15/1980`	`DOB:03/15/1980`	contained	0.85	match

`_char_spans(win, value)`: when the match is one OCR word that holds more than the value (`DOB:03/15/1980`), it returns which characters are the value (positions 4–14), so only those are replaced and `DOB:` stays.

Part 4 — Protecting printed form text

```
BOILERPLATE_PHRASES = ["HEALTH INSURANCE CLAIM FORM", "APPROVED BY NATIONAL
UNIFORM CLAIM COMMITTEE", "NUCC", "PICA",
    "MEDICARE", "MEDICAID", "TRICARE", "CHAMPVA", ..., "SSN", "EIN", "NPI",
..., "MM DD YY", "MM", "DD", "YY", ...,
    "UB-04", "NUBC", ...]

def boilerplate_word_ids(offset_map, lines=None):
    ids = set()
    for phrase in BOILERPLATE_PHRASES:
        for _score, win in match_text(phrase, offset_map, lines):
            ids |= {w['word_id'] for w in win}
    return ids
```

A fixed list of the printed text of the blank CMS-1500 and UB-04. Every place it appears on the page is protected, even if GPT forgot to list it. This is why the title and words like `MEDICARE` are never changed (a name detector might otherwise see `TRICARE` as a name).

`_printed_word_ids(printed, ...)` does the same for the `printed_text` lines GPT returned. A long line that does not match as a whole is matched in 3-word pieces, so one OCR misread does not lose the whole line.

Part 5 — Putting GPT's answer together

`vlm_pairs_and_hits(vlm, offset_map)`, for each field GPT returned:

```
lab_hits = match_text(label, offset_map, lines)
lab_win = lab_hits[0][1] if lab_hits else []
label_ids |= {w['word_id'] for w in lab_win}           # the label is printed
form text: protect it
...
val_hits = [(s, w) for s, w in match_text(value, offset_map, lines) if not
(... & lab_ids)]
if lab_win:                                             # the occurrence
closest to its label
    lx, ly = _centre(lab_win)
    win = min(val_hits, key=lambda h: (_centre(h[1])[0] - lx) ** 2 +
(_centre(h[1])[1] - ly) ** 2)[1]
else:
    win = val_hits[0][1]
pairs.append({"key_text": label, "key_word_ids": [...], "value_text": ...,
"value_word_ids": [...]})
kind = vlm_kind(f.get("category"), value)
if f.get("is_phi", True) and kind:
    hits.append({"word_ids": [...], "spans": _char_spans(win, value), "kind":
kind})
```

1. Find the label on the page; its words go into the protected set.
2. Find the value, ignoring any match that overlaps the label.
3. If the value appears more than once, take the one **nearest to its label** (distance between centres). Example: `SPRINGFIELD` is printed in box 5 (patient) and box 7 (insured); each GPT field is tied to the copy next to its own label.
4. Save the pair (for `ctx_text`) and, if GPT says it is PHI, a **hit** with its kind (for cell 12).

Then `free_text_phi` values become hits too (short values: best match only), and `printed_text` adds more protected words.

```
def vlm_build_context(image_path, offset_map, word_by_id):
    vlm = vlm_extract(image_path)
    pairs, hits, vlm_label_ids, stats = vlm_pairs_and_hits(vlm, offset_map)
    value_ids = {i for p in pairs for i in p['value_word_ids']} | {i for h in
hits for i in h['word_ids']}
    label_ids = ({i for p in pairs for i in p['key_word_ids']} |
vlm_label_ids) - value_ids
    ctx_t, ctx_m = build_context_view(pairs, [], word_by_id)
    return pairs, label_ids, ctx_t, ctx_m, hits, vlm_label_ids - value_ids,
stats

def protected_form_ids(offset_map, vlm_label_ids):
    return set(vlm_label_ids) | boilerplate_word_ids(offset_map)
```

`- value_ids` is a safety rule: if a word is both in a label and in a value, it is treated as a value, so real PHI is never protected by mistake. `protected_form_ids` adds the boilerplate list.

The run for this page

```
kv_pairs, label_word_ids, ctx_text, ctx_map = [], set(), "", []
vlm_hits, vlm_label_ids, vlm_stats = [], set(), {}
if USE_VLM:
    try:
        kv_pairs, label_word_ids, ctx_text, ctx_map, vlm_hits, vlm_label_ids,
vlm_stats = vlm_build_context(...)
    except Exception as exc:
        print(f"⚠ VLM step failed, continuing without label context:
{exc!r}")
vlm_label_ids = protected_form_ids(offset_map, vlm_label_ids)
```

Everything starts empty. If GPT fails, the page continues with empty context: Azure on `full_text`, the patterns and the boilerplate protection still run.

Example Stage 2 printout:

```
label                                    value               category
is_phi  matched_to_ocr
---------------------------------------  ------------------  --------------
------  --------------------------
1a. INSURED'S I.D. NUMBER                123456789           health_plan_id
True    yes: 123456789
2. PATIENT'S NAME (Last Name, First ...  SMITH, CAROLINE M   name
True    yes: SMITH, CAROLINE M
3. PATIENT'S BIRTH DATE                  03 15 1980          date
True    yes: 03 15 1980
SEX                                      F                   other
False   yes: F

protected (never changed) words: 212 -> 1a. BIRTH CHAMPVA CLAIM DD FORM
HEALTH ... MM ... YY
label: value lines sent to Azure: 31 | GPT PHI hits: 24
```

Cell 12: `detect_phi` — deciding which words are PHI

The idea: several independent detectors each suggest words. A word is PHI if **any** of them finds it, unless it is protected form text. A wrong extra flag only costs a harmless fake value; a miss leaks real data, so the code is deliberately generous.

#	Detector	Source name	Good at
1	Azure Tag on `full_text`	`azure`	Free text, notes, letters
2	Azure Tag on `ctx_text`	`azure_ctx`	Form values, now that each sits next to its label
3	Label keyword rules (`KEY_RULES`)	`rule`	Values under a label containing NAME, BIRTH, I.D., ZIP…
3b	GPT-5.4 hits (cell 11b)	`vlm`	Values GPT recognised visually
4	Regular expressions (`PATTERNS`)	`pattern`	Fixed formats: SSN, phone, e-mail, dates, ZIP
5	Propagation	`propagate`	The same name or ID printed again elsewhere

Settings at the top

```
MIN_AZURE_CONF = 0.0          # raise (e.g. 0.3) only if Azure over-tags your
documents
LABEL_OVERRIDE_CONF = 0.80    # a label/header word is replaced only if Azure
is at least this sure
SKIP_KINDS = set()            # kinds detected but left unchanged; empty =
replace everything
PROPAGATE_KINDS = {"name", "id", "phone", "email", "city"}
```

- `MIN_AZURE_CONF = 0.0`: every Azure entity is used, whatever its confidence.
- `SKIP_KINDS` is empty, so state, country and profession are replaced too (they used to be skipped).
- `PROPAGATE_KINDS`: kinds worth searching for again across the page. Dates are left out because the same date (a service date) can legitimately appear in many places and dates are already caught by patterns.

The lookup tables

```
CATEGORY_KINDS = [("patient", "name"), ("doctor", "name"), ("name", "name"),
("date", "date"), ("age", "age"),
    ("phone", "phone"), ("fax", "phone"), ("email", "email"), ("social",
"id"), ("street", "street"),
    ("city", "city"), ("state", "state"), ("zip", "zip"), ("country",
"country"), ("hospital", "org"),
    ("organization", "org"), ("profession", "profession"), ("location",
"street")]
```

Azure's category names (`Patient`, `Doctor`, `IDNum`, `HealthPlan`…) are matched by lower-case substring, first hit wins. `Patient` → name. Anything not listed (`IDNum`, `MedicalRecord`, `Account`…) → `id`. `category_to_kind` also applies the age rule: `Age 72` → not PHI, `Age 93` → PHI.

```
KEY_RULES = [
    (r"e-?mail", "email"),
    (r"phone|telephone|\btel\b|\bfax\b|mobile|\bcell\b", "phone"),
    (r"birth|\bdob\b|\bdates?
\b|admission|discharge|\bmm\b|\bdd\b|\byy\b|\byyyy\b", "date"),
    (r"\bssn\b|social security|\bi\.?
d\b|policy|member|subscriber|account|...", "id"),
    (r"\bzip\b|postal", "zip"),
    (r"address|street", "street"),
    (r"\bcity\b|\btown\b", "city"),
    (r"\bstate\b|province", "state"),
    (r"countr(y|ies)", "country"),
    (r"occupation|profession|job title|\bjob\b|designation", "profession"),
    (r"plan name|program
name|facility|employer|school|hospital|company|pay[eo]r", "org"),
    (r"\bname\b|signature|signed", "name"),
]
```

Each rule is a regular expression tested against a GPT label (lower case). `\b` means a word boundary, so `\bid\b` matches `I.D.` and `ID` but not the `id` inside `provider`. **Order matters**: `INSURED'S I.D. NUMBER` matches the id rule before it could match anything else, and `name` comes last because many labels contain the word name (`PLAN NAME` should be org, not a person).

```
FORM_TOKENS = {"MM", "DD", "YY", "YYYY", "CCYY", "MMDDYY", "MMDDYYYY", "M",
"F", "YES", "NO", "SELF",
               "SPOUSE", "CHILD", "OTHER", "NPI", "SSN", "EIN", "PICA",
"SOF", "N/A", "NA"}
STOP_VALUES = {"YES", "NO", "X", "M", "F", "NA", "N/A", "NONE", "SELF", ...,
"MALE", "FEMALE", "SEE", "ATTACHED"}
```

- `FORM_TOKENS` : printed captions that are **never changed, whatever any detector says**. This is the rule that keeps `MM DD YY` intact.
- `STOP_VALUES` : words that are never a PHI value even under a PHI label (a checkbox `X` , `SIGNATURE ON FILE`).

```
PATTERNS = [
    ("id",    re.compile(r"(?<!\d)\d{3}-\d{2}-\d{4}(?!\d)")),
# SSN 123-45-6789
    ("phone", re.compile(r"(?<!\d)(?:\(\d{3}\) ?|\d{3}[-. ])\d{3}[-. ]\d{4}
(?!\d)")), # (217) 555-0142
    ("email", re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}")),
    ("date",  re.compile(r"(?<!\d)\d{1,2}[/-]\d{1,2}[/-](?:\d{4}|\d{2})
(?!\d)")),  # 03/15/1980
    ("date",  re.compile(r"(?<!\d)(?:0[1-9]|1[0-2]) (?:0[1-9]|[12]\d|3[01])
(?:(?:19|20)\d{2}|\d{2})(?!\d)")),  # 03 15 1980
    ("date",  re.compile(r"\b" + _MONTHS + r" \d{1,2},? \d{4}\b", re.I)),
# March 5, 2021
    ("zip",   re.compile(r"(?<=\b[A-Z]{2} )\d{5}(?:-\d{4})?\b")),
# IL 62704
    ("zip",   re.compile(r"(?<=\b[A-Z]{2} )\d{9}\b")),
# IL 627041234
]
```

`(?<!\d)` and `(?!\d)` mean "no digit right before / after", so a 12-digit claim number is not read as an SSN. The space date pattern only accepts real months (01–12) and days (01–31), so a CPT code line like `99213 25 1` is not a date. The ZIP patterns only fire after a two-letter state and a space, so a random 5-digit amount is not a ZIP.

```
PRIORITY = {"pattern": 3, "azure": 2, "azure_ctx": 2, "vlm": 2, "rule": 1,
"propagate": 0}
```

When detectors disagree on the **kind** of a word, the one with the higher priority wins. A pattern is the most exact (an SSN pattern really is an SSN), propagation the least.

Helpers

- `_token_ok(kind, text)`: is this word a plausible value for the kind a label suggests? A name needs letters (≥ 2), a date needs a digit, an id needs a digit and ≥ 3 characters, a ZIP ≥ 5 digits, an e-mail an `@`. Example: under `PATIENT'S NAME`, the word `X` (a checkbox) is rejected.
- `_geo_kind(text)`: for a combined label `CITY STATE ZIP CODE`, decides word by word: ≥ 5 digits → zip, a known 2-letter state code → state, anything else → city. `SPRINGFIELD IL 62704` → city, state, zip.

- `_spans_to_words(start, end, entries, starts)`: turns a character range from Azure or a pattern into the OCR words it covers, with the part of each word that is inside. `bisect` finds the first candidate word by binary search instead of scanning every word.

`flag()` — every detection goes through here

```
def flag(wid, kind, source, conf=1.0, span=None):
    w = word_by_id.get(wid)
    if w is None or kind is None or kind in SKIP_KINDS:
        return                                                          # (1)
    if w['text'].upper().strip(".,:;()|[]") in FORM_TOKENS:
        blocked.append((wid, kind, source, conf, "form caption token"))
        return                                                          # (2)
    if wid in hard_labels and not (source == "pattern" and kind in ("id",
"phone", "email")):
        blocked.append((wid, kind, source, conf, "printed form text"))
        return                                                          # (3)
    if wid in label_word_ids:
        strong = (source == "pattern" and kind in ("id", "phone", "email"))
or \
                 (source.startswith("azure") and conf >= LABEL_OVERRIDE_CONF)
        if not strong:
            blocked.append((wid, kind, source, conf, "label word"))
            return                                                      # (4)
    n = len(w['text'])
    lo, hi = span if span else (0, n)
    f = flags.setdefault(wid, {"kind": kind, "prio": -1, "conf": 0.0,
"sources": set(), "mask": [False] * n})
    if PRIORITY[source] > f["prio"]:
        f["kind"], f["prio"] = kind, PRIORITY[source]                   # (5)
    f["conf"] = max(f["conf"], conf)
    f["sources"].add(source)
    for i in range(lo, hi):
        f["mask"][i] = True                                             # (6)
```

1. Unknown word, no kind (an age ⩽ 89) or a skipped kind: ignore.
2. A caption token (`MM`, `YES`, `NPI` …): never changed. Recorded in `blocked` for the Stage 3c printout.

3. **Printed form text** (GPT labels, GPT printed text, boilerplate): never changed. Only exception: an SSN / phone / e-mail pattern, because a real phone number cannot be form text.
4. A word in a label GPT paired: changed only with strong evidence (that same pattern exception, or Azure ≥ 0.80 sure).
5. The first detector creates the flag; a later detector with higher priority can change its kind.
6. `mask` marks which characters are PHI. Several detectors can each mark a part; the parts add up.

What happens to some words of the sample page:

Word	Detector	Check that decides	Result
`SMITH,`	azure, Patient 0.97, span 0–5	passes all checks	flagged name, mask `SMITH` (comma kept)
`MM`	rule (under BIRTH DATE)	(2) caption token	blocked
`TRICARE`	azure, Patient 0.41	(3) boilerplate	blocked
`NAME`	azure_ctx, Doctor 0.55	(4) label word, 0.55 < 0.80	blocked
`(217)` `555-0142`	pattern phone	passes	flagged phone
`F` (sex box)	vlm says not PHI; azure none	never suggested	unchanged

The detectors in order

1 + 2. Azure on both texts. For each entity: skip if below `MIN_AZURE_CONF`, turn the category into a kind, map the character range to words (`_spans_to_words`), `flag` each word with its part. In the `ctx_text` view the offsets go through `ctx_map`, so only value words can be hit.

3. Label rules. For each GPT pair, `rule_kinds(label)` lists the kinds whose regex matches the label. If the label names two or more of city / state / zip, each word gets its kind from `_geo_kind`; otherwise the first kind is used. `_token_ok` must agree.

Example: label `5. PATIENT'S ADDRESS (No., Street)` → street; value `482 MAPLE AVE` → all three words flagged street. Label `CITY STATE ZIP CODE` with value `SPRINGFIELD IL 62704` → city / state / zip.

3b. GPT hits. Each GPT PHI value, already on its OCR words, is flagged with source `vlm` and confidence 0.9. Two filters: stop values are skipped, and a name hit must contain a letter.

4. Patterns run on `full_text`; every match is flagged with confidence 1.0.

5. Propagation.

```
seeds = {}
for wid, f in flags.items():
    if f["kind"] not in PROPAGATE_KINDS:
        continue
    for m in re.finditer(r"[A-Za-z0-9]+", word_by_id[wid]['text']):
        if not all(f["mask"][m.start():m.end()]):
            continue
        tok = m.group().upper()
        is_num = any(ch.isdigit() for ch in tok)
        if tok in STOP_VALUES or len(tok) < (4 if is_num else 3):
            continue
        seeds.setdefault(tok, f["kind"])
for e in offset_map:
    for m in re.finditer(r"[A-Za-z0-9]+", e['text']):
        kind = seeds.get(m.group().upper())
        if kind: ... flag(e['word_id'], kind, "propagate", 1.0, (m.start(),
m.end()))
```

Every fully-flagged name / id / phone / e-mail / city token becomes a "seed". Short tokens are left out (letters < 3, numbers < 4) so `M` or `12` are not spread across the page. Then every OCR word is searched for the seeds. Example: `SMITH` from box 2 is a seed, so `SMITH` in the signature line `C. SMITH` (box 12) is flagged too, even if no other detector saw it.

6. Credentials after a name. `MD`, `PA`, `DO`… are also state codes. If one was flagged as a state, is right after a name on the same line, and did not come from a STATE label, its flag is removed. `JOHN SMITH MD` → `MD` stays; `IL` under the STATE label is still replaced.

Grouping into entities

```
for e in offset_map:
    f = flags.get(e['word_id'])
    if not f: continue
    last = entities[-1] if entities else None
    if last and last["kind"] == f["kind"] and last["line_idx"] ==
e['line_idx'] \
            and last["words"][-1]['idx'] == e['idx'] - 1:
        last["words"].append(e)
    else:
        entities.append({"kind": f["kind"], "line_idx": e['line_idx'],
"words": [e]})
```

Neighbouring flagged words of the same kind on the same line form one **entity**: `SMITH,` `CAROLINE` → one name, `482` `MAPLE` `AVE` → one street. Cell 13 makes one fake value per entity, so a full name gets a consistent fake.

Watch out — a one-letter middle initial. `M` and `F` are in `FORM_TOKENS` (they are the SEX checkbox captions), so `flag()` blocks them everywhere. On the sample page the entity is therefore `SMITH, CAROLINE` and the middle initial `M` stays as printed. One letter is not an identifier on its own, so this is accepted as a trade-off. If you want initials replaced, remove `M` and `F` from `FORM_TOKENS` and rely on `BOILERPLATE_PHRASES` / GPT's `printed_text` to protect the SEX captions instead.

Outputs:

```
flags   = {"7c2f...e91": {"kind": "name", "prio": 2, "conf": 0.97,
                          "sources": {"azure", "azure_ctx", "vlm", "rule"},
                          "mask": [True, True, True, True, True, False]}, ...}
entities = [{"kind": "name", "line_idx": 7, "words": [<SMITH,>, <CAROLINE>],
"original": "SMITH, CAROLINE", "sources": ["azure", "azure_ctx", "rule",
"vlm"], "conf": 0.97}, ...]
```

Example summary printout:

```
Azure entities returned: 41  (line view: 19, label:value view: 22)
PHI words flagged: 38  |  entities: 17
By kind:   {'id': 3, 'name': 4, 'date': 3, 'street': 1, 'city': 1, 'state':
1, 'zip': 1, 'phone': 1, 'org': 2}
By source: {'azure': 12, 'azure_ctx': 15, 'rule': 13, 'vlm': 16, 'pattern':
5, 'propagate': 2}
Protected words a detector tried to change (stopped): 14
```

Cell 13: surrogates — how each fake value is made

Two rules drive this cell:

1. **Same shape.** The fake has the same number of characters, the same upper / lower case, digits stay digits and punctuation stays in place. The new text then fits in the old box without shrinking.
2. **Repeatable.** The same original always gives the same fake (on this page, on other pages and on later runs), so a name printed in three boxes gets the same fake name three times.

All examples below were produced by running the cell's real code with `DEID_SURROGATE_KEY = "demo-key-2026"`. Your key gives different fakes.

Part 1 — The repeatable random generator

```
SURROGATE_KEY = (os.environ.get("DEID_SURROGATE_KEY") or
os.urandom(16).hex()).encode()

def _rng(*parts):
    digest = hmac.new(SURROGATE_KEY, "|".join(parts).encode(),
hashlib.sha256).digest()
    return random.Random(int.from_bytes(digest[:8], "big"))

DATE_SHIFT_DAYS = -(1 + _rng("date-shift").randrange(364))   # same shift for
every date
```

Line	What it does
`SURROGATE_KEY`	A secret from `.env`. Without it a random key is made, valid only until the notebook restarts (fakes then differ between runs).

Line	What it does
`hmac.new(key, "name|SMITH", sha256)`	Mixes the secret with the original value into a 32-byte fingerprint. Same key + same value → same fingerprint, always. Without the key nobody can work back to `SMITH`.
`digest[:8]` → `int`	The first 8 bytes become one big number, the **seed**.
`random.Random(seed)`	A random generator that, started from the same seed, makes the same "random" choices every time.
`DATE_SHIFT_DAYS`	One number from −1 to −364, chosen once from the key. Every date is moved back by this many days.

Example: `_rng("name", "SMITH")` → fingerprint starts `0f43cf60a5f050a4` → seed `1099950748016332964` → always the same choice below. With this key `DATE_SHIFT_DAYS = -126`.

Part 2 — The word pools

```
_NAMES = """LI WU NG ... LEE KIM RAY ... SMITH STONE ... DIANA EMILY ...
CAROLINE ... REGINALD ..."""
_PLACES = """ELM OAK ASH ... MAPLE RIDGE GROVE ... SPRINGFIELD ...
BLOOMINGTON ..."""

def _pool(words):
    by_len = {}
    for w in words.split():
        by_len.setdefault(len(w), []).append(w)
    return by_len

POOLS = {"name": _pool(_NAMES), "place": _pool(_PLACES)}
```

The names and place names are grouped **by length**: `POOLS["name"][5]` is every 5-letter name (`ADAMS ALLEN BAKER … DIANA …`). That is how a 5-letter name always gets a 5-letter fake.

```
KEEP = {"name":  {"JR", "SR", "II", "III", "MD", "DO", "RN", "DR", "MR",
"MRS", ...},
        "place": {"RD", "ST", "AVE", "BLVD", "APT", "PO", "BOX", "N", "S",
"HOSPITAL", "CLINIC", ...}}
```

Words that identify nobody and are kept: titles and credentials in names, street types and directions in addresses. `482 MAPLE AVE` keeps `AVE`.

```
_UPPER_W = dict(zip("ABC...", [667, 667, 722, ...]))      # letter widths of
a typical sans font
def _nominal_width(s):
    return sum(_UPPER_W.get(c) or _LOWER_W.get(c) or 556 for c in s)
```

In a proportional font `WWWWW` is much wider than `IIIII`. These are standard letter widths (per 1000 units). The code prefers fakes about as wide as the original, so spacing on the line still looks natural.

Part 3 — Replacing letters, digits and date pieces

```
def _fake_alpha(group, tok):
    up = tok.upper()
    if up in KEEP.get(group, ()):
        return tok                                                    # (1)
    key = (group, up)
    if key not in _memo:
        rng, n = _rng(group, up), len(up)
        if n == 1:
            fake = rng.choice([c for c in "ABCDEFGHJKLMNPRSTW" if c != up])
# (2)
        else:
            pool = [p for p in POOLS.get(group, {}).get(n, []) if p != up]
            if pool:
                target = _nominal_width(tok)
                pool.sort(key=lambda p: (abs(_nominal_width(_match_case(p,
tok)) - target), p))
                fake = rng.choice(pool[:5])                           # (3)
            elif group == "shape" and n < 3:
                fake = "".join(rng.choice("ABCDEFGHJKLMNPRSTW") for _ in
range(n))
```

```
            else:
                fake = _pronounceable(n, rng)                          # (4)
        _memo[key] = fake
    return _match_case(_memo[key], tok)                                # (5)
```

1. Kept words (AVE, MD…) are returned unchanged.
2. A single letter becomes another letter.
3. **The main rule:** take all pool words of the same length (except the original), sort them by how close their width is to the original's, keep the 5 closest, and let the seeded generator pick one.
4. No pool word that long (a 14-letter surname): build a pronounceable fake by alternating consonants and vowels (`KATOMERU…`).
5. `_memo` remembers the choice for this run; `_match_case` copies the original's case letter by letter (`Smith` → `Diana`).

Worked example: `SMITH`

Step	Value
Length	5 → pool `POOLS["name"][5]` (without SMITH)
Width of `SMITH`	667+833+278+611+722 = 3111
5 closest in width	LEWIS 3112, KELLY 3113, BRIAN 3056, DAVID 3056, DIANA 3056
Seeded pick	**DIANA**
On the page	the OCR word is `SMITH,` ; only the masked core `SMITH` is replaced → `DIANA,`

```
def _fake_digits(group, tok):
    ...
    for _ in range(5):
        fake = "".join(rng.choice("0123456789") for _ in tok)
        if fake != tok:
            break
```

Each digit run becomes the same number of random (seeded) digits; it retries if by chance it drew the original. `123456789` → `407350336` .

```
def _fake_date_fragment(tok):
    n, v = len(tok), int(tok)
    if n == 4 and tok[:2] in ("19", "20"):
        return tok                      # a year on its own is not
identifying
    ...
    choices = [c for c in (range(1, 10 if n == 1 else 13) if v <= 12 else
range(13, 29)) if c != v]
    return str(rng.choice(choices)).zfill(n)
```

For a date piece printed alone in a box, used only when the whole date could not be put together. A value ⩽ 12 could be a month, so it gets another value 1–12; a value 13–31 must be a day, so it gets 13–28 (valid in every month). `zfill` keeps the leading zero: `03` → `11`, `15` → `22`, `1980` stays.

Part 4 — Dates: shifted, not randomised

```
_DATE_FORMATS = ["%m/%d/%Y", "%m/%d/%y", "%m-%d-%Y", "%m %d %Y", "%m %d %y",
"%Y-%m-%d", "%B %d, %Y", ..., "%m%d%Y"]

def shift_date_string(s):
    for fmt in _DATE_FORMATS:
        try:
            d = datetime.strptime(s, fmt)          # does the text read as a
date in this format?
        except ValueError:
            continue
        if not 1900 <= d.year <= 2100:
            continue
        shifted = d + timedelta(days=DATE_SHIFT_DAYS)
        out = shifted.strftime(fmt)                # written back in the SAME
format
        ...
        return out
    return None
```

The text is tried against each format until one reads it as a real date. The date is moved by `DATE_SHIFT_DAYS` and written back in the same format; month names keep a similar length and an original without leading zeros gets none.

Why shift instead of random dates: all dates move together, so the time between them stays true (a patient's age, the length of a hospital stay, the order of service dates), and the data is still useful for analysis.

Original	Shift −126 days	Fake
`03/15/1980`	15 Mar 1980 − 126 days = 10 Nov 1979	`11/10/1979`
`03 15 1980`	same date, space format	`11 10 1979`
`03 15 80`	two-digit year kept	`11 10 79`

`_merge_split_dates(entities)`: on a CMS-1500 the birth date is printed as three separate words in three boxes (`03` `15` `1980`). This function finds three single-number date entities on the same row (centres within 0.6 × line height), left to right, close together, with sizes 2 / 2 / 2-or-4, and shifts them **as one date**. So `03` `15` `1980` becomes `11` `10` `1979`, the same as `03/15/1980` written elsewhere becomes `11/10/1979`. Without it each piece would be faked alone and the two copies of the birth date would disagree.

Part 5 — State, country, profession: real values

```
PHRASE_POOLS = {"state": sorted(STATE_CODES) + _phrases(_STATE_NAMES),
                "country": _phrases(_COUNTRIES), "profession":
_phrases(_PROFESSIONS)}

def phrase_surrogate(kind, cores):
    ...
    if kind == "state" and len(cores) == 1 and len(up) == 2:
        pool = sorted(STATE_CODES)                    # a code stays a code
    cands = [p for p in pool if len(p.split()) == len(cores) and p != up]
    cands.sort(key=lambda p: (abs(len(p) - target), p))
    _memo[key] = _rng("phrase", kind, up).choice(cands[:5])
```

A made-up state like `KT` would look wrong, so these get a **real** value: same number of words, length close to the original (5 closest), seeded pick, case copied.

Original	Fake
`IL`	`AL`

Original	Fake
`NEW YORK`	`SOUTH DAKOTA`
`TEACHER`	`LABORER`

Part 6 — Putting it together

```
def surrogate_core(kind, core):
    if kind == "date":
        shifted = shift_date_string(core)
        if shifted is not None:
            return shifted
    group = "name" if kind == "name" else "place" if kind in ("street",
"city", "org", "state", "country") else "shape"
    out = []
    for run in re.findall(r"[A-Za-z]+|\d+|[^A-Za-z\d]+", core):
        if run[0].isalpha():
            out.append(run if kind == "date" or (kind == "email" and
run.lower() in _TLDS) else _fake_alpha(group, run))
        elif run[0].isdigit():
            out.append(_fake_date_fragment(run) if kind == "date" else
_fake_digits(group, run))
        else:
            out.append(run)                    # punctuation and spaces stay
    return "".join(out)
```

The value is cut into runs of letters, digits and other characters. `(217) 555-0142` → `(` `217` `)` ` ` `555` `-` `0142`: digits are replaced, brackets, space and dash stay, so the fake still looks like a phone number. Names use the name pool, addresses / cities / organisations the place pool, everything else ("shape") keeps only its pattern.

```
def build_surrogates(flags, entities):
    for ent in entities:
        ent["parts"] = []
        for w in ent["words"]:
            mask = flags[w['word_id']]["mask"]
            a = mask.index(True)
            b = len(mask) - mask[::-1].index(True)
            ent["parts"].append((w['text'][:a], w['text'][a:b], w['text']
[b:]))   # prefix, PHI core, suffix
        ent["new_cores"] = None
    _merge_split_dates(entities)
    for ent in entities:
        # 1. multi-word date shifted as one   2. state/country/profession
phrase   3. surrogate_core per word
        ...
        for w, (pre, _core, suf), new_core in zip(ent["words"], parts,
new_cores):
            w["new_text"] = pre + new_core + suf
        ent["surrogate"] = " ".join(w["new_text"] for w in ent["words"])
```

Each word is split by its mask into **prefix + PHI core + suffix**. `SMITH,` → (`""`, `SMITH`, `,`); `DOB:03/15/1980` → (`DOB:`, `03/15/1980`, `""`). Only the core is replaced, so `new_text` is `DIANA,` and `DOB:11/10/1979`. Every flagged word now has `new_text`, and every entity has `surrogate`.

Result for the sample page

Kind	Original	Fake	Method
id	`123456789`	`407350336`	digits replaced
name	`SMITH, CAROLINE`	`DIANA, REGINALD`	name pool, same length, closest width
date	`03` `15` `1980`	`11` `10` `1979`	split date merged, shifted −126 days
street	`482 MAPLE AVE`	`639 GROVE AVE`	digits replaced, place pool, AVE kept
city	`SPRINGFIELD`	`BLOOMINGTON`	place pool (11 letters)
state	`IL`	`AL`	real state code
zip	`62704`	`86762`	digits replaced

Kind	Original	Fake	Method
phone	`(217) 555-0142`	`(045) 623-7686`	digits replaced, punctuation kept
email	`c.smith@mail.com`	`f.akiwi@acif.com`	letters replaced, `.com` kept

Stage 4 printout:

```
kind    original           fake               found_by
------  -----------------  -----------------  ----------------------------
id      123456789          407350336          azure_ctx,rule,vlm
name    SMITH, CAROLINE    DIANA, REGINALD    azure,azure_ctx,rule,vlm
date    03                 11                 azure,rule,vlm
...
```

Cell 14: `render_page` — drawing the fakes on the image

The principle is "fit on the original, then swap the text". For every PHI word we know its real text (from OCR) and its real pixels. So the code draws the **original** text in each candidate font, compares that with the real pixels, and the font and size that reproduce the original best are used for the fake.

Settings

```
MIN_OCR_CONF = 70.0     # below this the OCR text is not trusted
MAX_SKEW_DEG = 3.0      # words rotated more than this
ROTATE_MIN_DEG = 0.7    # words rotated more than this are drawn rotated
CALIB_WORDS = 60        # extra non-PHI words sampled to learn the fonts on
the page
FONT_PEER_WEIGHT = 0.15 # how strongly same-size text elsewhere settles a
near-tie between fonts
BLUR_RADIUS = 0.45      # softening of the drawn text (0 = crisp)
BOX_COLOR = (255, 0, 0) # colour of the fail-safe box
EXTRA_FONT_PATHS = []   # extra .ttf/.otf files to consider
KEEP_LOW_CONFIDENCE = False
```

Note on the fail-safe box. The comment at the top of the cell says handwriting, low-confidence and rotated words are covered with a solid box. In the current code that path is switched off: the `boxes` list is never filled (`#boxes.append(w)` is commented out). The `reason` is still worked out, but it is only used for `KEEP_LOW_CONFIDENCE`. So today **every**

PHI word whose fake differs is erased and redrawn in a printed font, handwriting and tilted words included. If you want the red box back for handwriting, put `boxes.append(w); continue` in place of the `reason` checks.

Part 1 — Fonts and text rendering

```
FONT_DIRS = [r"C:\Windows\Fonts", ..., "/usr/share/fonts", "/Library/Fonts",
...]
FONT_FILES = ["cour.ttf", "courbd.ttf", "consola.ttf", "lucon.ttf",
"OCRAEXT.TTF",
              "arial.ttf", "arialbd.ttf", "arialn.ttf", "calibri.ttf",
"verdana.ttf", "tahoma.ttf",
              "times.ttf", "timesbd.ttf", "LiberationMono-Regular.ttf", ...]

def find_fonts():
    # walks the font folders, keeps the files named in FONT_FILES (in that
order) + EXTRA_FONT_PATHS
```

The candidates cover the fonts claim forms are usually printed in: typewriter (Courier, Consolas, OCR-A), sans (Arial, Calibri, Verdana), narrow (Arial Narrow) and serif (Times), regular and bold. The Liberation fonts are free look-alikes with the same letter widths, for Linux and Mac.

```
@lru_cache(maxsize=100000)
def render_text(text, path, size):
    font = _font(path, size)
    l, t, r, b = font.getbbox(text, anchor="ls")
    ...
    ImageDraw.Draw(im).text((pad - l, pad - t), text, font=font, fill=255,
anchor="ls")
    a = np.asarray(im)
    ys, xs = np.where(a >= 96)
    x0, x1, y0, y1 = xs.min(), xs.max() + 1, ys.min(), ys.max() + 1
    return a[y0:y1, x0:x1].copy(), int(x0 - (pad - l)), int(y0 - (pad - t))
```

Draws `text` white-on-black in a small image and crops it to the ink. It returns the ink picture plus where the ink starts relative to the pen position and the **baseline** (`anchor="ls"` = left, on the baseline). The baseline is needed to put the fake on the same line as the original. `lru_cache` remembers results: the same word in the same font and size is drawn only once.

Part 2 — Reading the original pixels

```
def _otsu(values):   # picks the grey level that best separates dark ink from
light paper
```

Otsu's method: it tries every grey level 0–255 as a dividing line and keeps the one where the two groups (ink / paper) are most clearly different. Works on white, grey and coloured forms without a fixed number.

```
def analyze_word(gray, w, pad=2):
    x0, y0, x1, y1 = w['px']
    crop = gray[Y0:Y1, X0:X1]                          # the word box + 2 px
    if w.get('poly') and abs(w['angle']) > 0.2:
        ...polygon...                                   # tilted word: only
pixels inside its rotated box
    if int(crop.max()) - int(crop.min()) >= 40:         # enough contrast to
have ink
        dark = crop <= _otsu(crop)
        lines = _find_form_lines(dark)
        ink = dark & ~lines & inside
    return {"origin": (X0, Y0), "ink": ink, "lines": lines, "inside": inside,
"inkbox": inkbox}
```

- Crops the word's pixel box from cell 6 (plus 2 px).
- Dark pixels = ink. But the crop also holds parts of the **form's ruled lines**.
- `_find_form_lines`: a form line enters at one edge of the crop and leaves at the opposite edge, and is covered for at least 92% of its length; letters never do that. Those pixels are marked as `lines` (horizontal, and vertical by running the same test on the turned crop). They are removed from the ink, so they are never erased.
- `inkbox`: the tight box around the real ink.

`_tight_mask` crops the ink tight and, if the word is tilted, rotates it straight so it can be compared with straight rendered text.

Part 3 — Choosing the font and size

```
def fit_size(text, path, ink_w, ink_h):
    ref = render_text(text, path, 64)
    by_width = len(text.strip()) >= 3
    guess = 64 * (ink_w / rw if by_width else ink_h / rh)
    # tries guess-1, guess, guess+1 and keeps the size whose width (or
height) is closest
```

Draws the original text at size 64, then scales: if `SMITH,` is 141 px wide on the page and 282 px wide at size 64, the size is about 32. It checks 31, 32, 33 and keeps the closest. Words of 1–2 characters are fitted by height, since their width says little.

```
def score_fit(text, path, size, tight):
    rendered = _soft(resize(render_text(text, path, size), to the real ink's
size))
    target = _soft(tight)
    dice = 2 * np.minimum(rendered, target).sum() / (rendered.sum() +
target.sum() + 1e-6)
    shape_penalty = (height agreement) * (width agreement)   # 1 px tolerance
    return dice * shape_penalty
```

Dice score: put the drawn letters on top of the real letters; 2 × overlap ÷ (ink of both). 1.0 = identical shapes, 0 = no overlap. Both are slightly blurred first (`_soft`) so a 1-pixel offset does not ruin the score. The **shape penalty** lowers the score when the drawn word's height or width differs from the real one by more than 1 px, so a font that only matches after stretching loses.

Illustration for the word `SMITH,` (scores are examples of the kind of numbers you see):

Font	Fitted size	Dice	Shape penalty	Score
Arial	32	0.90	0.98	**0.88**
Arial Bold	31	0.83	0.95	0.79
Courier New	30	0.76	0.93	0.71
Times New Roman	33	0.70	0.89	0.62

```
def choose_styles(gray, offset_map, phi_word_ids):
    calib = words on the PHI lines + up to 60 other clean words     #
_fittable: printed, straight, conf >= 90, >= 3 letters
    for w in calib:  fits[w] = {font: (score_fit, fit_size)} for every font
    # Pass 1: lines with >= 5 letters
    font = max(own, key=lambda f: own[f] + FONT_PEER_WEIGHT * votes.get(f, 0)
/ len(peers))
    # Pass 2: lines that are mostly digits take the font of the nearest
lettered line of the same size,
    #         if that font scores within 0.08 of this line's own best
    main_font = the font used most on the PHI lines
```

- The font is chosen **per line**, so a form with typed values in Courier and a printed header in Arial works.
- A line's score is the average over its words. Small near-ties are settled by **peer votes**: which font wins most often among other words of a similar size on the page (size buckets 15% apart), weighted 0.15.
- Digits look alike in many fonts, so a line like `62704` borrows the font of the nearest lettered line of the same size, if that font also fits it well.
- Size = the median fitted size of the line, so neighbouring words never differ.

`style_for_word` gives each PHI word its line's font and size; a word that clearly differs in width (> 25%) gets its own size; a word on a line with no style takes the nearest line whose style reproduces its ink height.

Part 4 — Erasing and drawing

```
def erase_word(out, orig, info, box, rng, page_bg):
    mask = _dilate(info["ink"])                          # the ink + 1 px (the
grey anti-aliased edge)
    mask &= allowed & ~info["lines"] & info["inside"]   # inside the word
box, never a form line
    background = ~_dilate(_dilate(info["ink"])) & ~_dilate(info["lines"])
    samples = region_orig[background]                    # real paper pixels
next to the word
    out[...][mask] = samples[rng.integers(0, len(samples), int(mask.sum()))]
```

Only ink pixels are erased, never the ruled lines. They are filled with **real background pixels picked at random from around the word**, not plain white, so on grey or speckled scans the patch is invisible. `rng` is seeded with 0, so the result is the same each run.

```
def ink_colour(orig, info, ref):
    bg = np.median(region[background], axis=0)
    mass = np.clip(bg - region[near], 0, None).sum(axis=0)     # total
darkness of the original word
    coverage = ref[0].astype(float).sum() / 255.0               # how much
ink the same word has when redrawn
    return np.clip(bg - mass / max(coverage, 1.0), 0, 255), bg
```

Scanned strokes are thin and grey, so the darkest pixel would be too dark and the average too light. This picks the colour that gives the redrawn word the **same total amount of darkness** as the original.

`word_geometry` finds where the original ink starts and ends and computes the **baseline** from the ink box and the rendered text's offset. `draw_word` then:

1. renders `new_text` in the chosen font and size;
2. squeezes it horizontally if it would run into the next word (`replaced (condensed)`), rather than making the font smaller;
3. rotates it by the word's angle if it is tilted more than 0.7°;
4. blurs it by 0.45 px so it looks scanned, not digital;
5. blends it into the image: `pixel = old × (1 − alpha) + colour × alpha`.

Part 5 — `render_page`, the whole page

```
phi = [w for w in offset_map if w['word_id'] in flags]
doc_font, line_styles = choose_styles(gray, offset_map, set(flags))
for w in phi:
    if w.get('new_text', w['text']) == w['text']:
        actions[...] = "unchanged"; continue          # nothing to change
    ...font, size, baseline, colour...
    draws.append({...})
# runs of neighbouring replaced words keep the ORIGINAL gaps between them;
squeeze or spread to fit
for d in draws: erase_word(...)                         # erase everything
first
for d in draws: actions[...] = draw_word(...)           # then draw
```

- Neighbouring replaced words (`DIANA`, `REGINALD`) are laid out as one run: the first starts where the original started, the original gaps are kept, and the whole run is squeezed only if it would reach the next untouched word.

- **All words are erased before any is drawn**, so one word's eraser never wipes another's new text.
- A grey scan is saved as grey. The output is saved as a new PNG built from pixels only, so no metadata from the original file is carried over.

Example printout:

```
Main font used for the replacements: arial.ttf
Words: {'replaced': 35, 'unchanged': 3}
Saved: ...\output_images\CMS 1500 202604230008171_page_1_deid.png

STAGE 5 - Image: action per PHI word
original     drawn        action
-----------  -----------  --------------------
123456789    407350336    replaced
SMITH,       DIANA,       replaced
CAROLINE     REGINALD     replaced
SPRINGFIELD  BLOOMINGTON  replaced (condensed)
1980         1979         replaced
```

`unchanged` words are flagged words whose fake equals the original (for example a year on its own, or `AVE`).

Batch cells: running every page and the files you get

Cells 1–14 run the pipeline once for `IMAGE_PATH`, for testing. The last three cells run it for every image in a folder.

Batch cell 1 — settings and helper functions

```
BATCH_INPUT_DIR = Path(r"C:\...\Input_docs\data")
BATCH_COMPARISON_DIR = Path(r"C:\...\Input_docs\compared_data")
BATCH_RECURSIVE = True                      # also look in sub-folders
SAVE_VERIFICATION_RECORD = True
VERIFY_DIR = Path(r"C:\...\Input_docs\verification_records")
```

`discover_images(root_dir, recursive)`

```
for path in root_dir.glob("**/*" if recursive else "*"):
    if path.parent.name.lower() == "output_images":           continue   #
our own outputs
    if path.stem.lower().endswith(("_deid", "_comparison")): continue   # our
own outputs
    if path.suffix.lower() in SUPPORTED_IMAGE_FORMATS:
images.append(path)
return sorted(images)
```

Finds every image, skipping the notebook's own outputs so a second run does not de-identify the de-identified images. `sorted` gives the same order every run.

`save_comparison_image` pastes the original and the de-identified image side by side on a white canvas (24 px gap, 48 px band on top) with the labels "Original" and "De-identified".

`_ocr_bytes_for(image)`: the verification step OCRs the **output** image again. A PNG of a JPG scan can be bigger than the 10 MB Textract limit, so if it is, a high-quality JPEG (95) is sent for that OCR call only.

`save_verification_record`

```
deid_blocks = get_ocr(_ocr_bytes_for(deid_image))["Blocks"]          # second
Textract call: read the OUTPUT
_deid_text, deid_words = build_word_index(deid_blocks, deid_image.width,
deid_image.height)
record = {
  "image_path": ..., "deid_image_path": ..., "image_size": [w, h],
  "original_words": [{word_id, idx, line_idx, text, px, confidence,
handwriting,
                      flagged, kind, sources, new_text, action, entity} for
every OCR word],
  "entities":       [{n, kind, original, surrogate, sources, word_ids}],
  "deid_words":     [{idx, line_idx, text, px, confidence} for every word
read from the output],
}
```

It records, for every word, what was on the original, whether it was flagged and why, what was drawn, and what Textract can read at that spot on the output. Nothing is judged here; `phi_leak_verification_14_oct.ipynb` reads these files and checks, position by position, whether original PHI text is still readable. It costs one extra Textract call per page.

Example entry for `SMITH,`:

```
{"word_id": "7c2f...e91", "idx": 9, "line_idx": 7, "text": "SMITH,", "px":
[109, 529, 250, 569],
 "confidence": 99.4, "handwriting": false, "flagged": true, "kind": "name",
 "sources": ["azure", "azure_ctx", "rule", "vlm"], "new_text": "DIANA,",
"action": "replaced", "entity": 1}
```

This file holds the real value next to its fake, so it is a **re-identification key**: keep `verification_records` as protected as the original images.

`process_image_pipeline(image_path, batch_input_dir, comparison_root)`

The same steps as cells 3–14, in one function, for one page:

```
image_path = Path(load_image(str(image_path)))
response = get_ocr(image_bytes)                                          #
cell 4
full_text, offset_map = build_word_index(blocks, img_w, img_h)           #
cell 6
if USE_VLM:
    try:
        kv_pairs, label_word_ids, ctx_text, ctx_map, vlm_hits, vlm_label_ids,
vlm_stats = vlm_build_context(...)
    except Exception as exc:
        vlm_stats = {"vlm_error": repr(exc)[:200]}                       #
cell 11b, page continues
vlm_label_ids = protected_form_ids(offset_map, vlm_label_ids)
flags, entities, azure_entities = detect_phi(...)                        #
cell 12
build_surrogates(flags, entities)                                        #
cell 13
deid_image, actions, doc_font = render_page(str(image_path), offset_map,
flags)   # cell 14
# save <stem>_deid.png, <stem>_comparison.png, <stem>_verify.json
return {... one summary row ...}
```

- A GPT failure is caught and written into `vlm_error`; the page still runs without label context.
- `relative_dir` keeps the input's sub-folder structure in the comparison and verification folders (`data\clinicA\page1.png` → `compared_data\clinicA\page1_comparison.png`).
- `DEBUG_BATCH` prints all five stages for each page.

Batch cell 2 — the loop

```
for index, image_path in enumerate(batch_images, start=1):
    try:
        result = process_image_pipeline(image_path, BATCH_INPUT_DIR,
BATCH_COMPARISON_DIR)
        batch_results.append(result)
    except Exception as exc:
        batch_errors.append({"image_path": str(image_path), "error":
repr(exc)})
# writes batch_summary.csv (one row per page) and batch_errors.csv (if any)
```

One page failing (a corrupt file, a Textract error) is logged and the loop moves on. At the end, `csv.DictWriter` writes all rows to `batch_summary.csv`, and errors to `batch_errors.csv`.

Example console output:

```
Found 3 image(s) under C:\...\Input_docs\data
[1/3] Processing CMS 1500 202604230008171_page_1.png
    saved deid=...\output_images\CMS 1500 202604230008171_page_1_deid.png |
comparison=... | phi_entities=17 | replaced_words=35
[2/3] Processing UB04_0042_page_1.png
    ⚠ VLM step failed for UB04_0042_page_1.png, continuing without label
context: APITimeoutError(...)
    saved deid=... | phi_entities=22 | replaced_words=41
[3/3] Processing scan_bad.png
    FAILED: AssertionError('Image exceeds 10MB sync limit: scan_bad.png')
Completed: 2 image(s)
Failed: 1 image(s)
```

Example row of `batch_summary.csv` (main columns):

Column	Example	Meaning
`phi_words` / `phi_entities`	38 / 17	PHI found
`azure_entities`	41	entities Azure returned (both views)
`replaced_words`	35	redrawn (including condensed)
`unchanged_words`	3	flagged but the fake equals the original (`AVE`, a lone year)

Column	Example	Meaning
`boxed_words`	0	always 0 while the box path is off (cell 14)
`kept_low_confidence_words`	0	only above 0 if `KEEP_LOW_CONFIDENCE = True`
`font_used`	arial.ttf	main font on the PHI lines
`context_source` / `vlm_used`	vlm / True	`none` / False when GPT failed
`vlm_fields` / `vlm_fields_matched`	31 / 30	fields GPT returned / found among OCR words
`vlm_phi_words` / `vlm_only_phi_words`	26 / 2	words GPT helped flag / words only GPT found
`protected_words_blocked`	14	attempts to change printed form text that were stopped
`vlm_error`	(empty)	why GPT failed on that page

Batch cell 3 — preview

Shows one comparison image with matplotlib. It uses `plt`, which is not imported anywhere in the notebook, so add `import matplotlib.pyplot as plt` at the top of this cell or it stops with `NameError`.

One value end to end: `SMITH` from scan to fake

The whole notebook, followed for one word of the sample page.

Step	Function (cell)	What happens to `SMITH`	Data after the step
1	`get_ocr` (4)	Textract reads the word `SMITH,` at Left 0.0431, Top 0.1604	a WORD block with id `7c2f...e91`

Step	Function (cell)	What happens to `SMITH`	Data after the step
2	`build_word_index` (6)	Placed in the page text at 212–218 and converted to pixels	`px (109, 529, 250, 569)`, `angle 0.14`, `line_idx 7`
3	`vlm_extract` (11b)	GPT-5.4 returns label `2. PATIENT'S NAME …`, value `SMITH, CAROLINE M`, category `name`, PHI	JSON field
4	`match_text` (11b)	Value found on OCR line 7 with score 1.00, closest to its label	value words `SMITH, CAROLINE M`
5	`vlm_pairs_and_hits` (11b)	Becomes a pair (for Azure) and a hit of kind `name`; the label words become protected	`kv_pairs`, `vlm_hits`, `vlm_label_ids`
6	`build_context_view` (8)	Written as `2. PATIENT'S NAME (…): SMITH, CAROLINE M`	`ctx_text`, `ctx_map` (SMITH, → offsets 98–104)
7	`azure_tag` (11) on `full_text`	Azure: `Patient`, offsets 212–217, confidence 0.97	entity
8	`azure_tag` (11) on `ctx_text`	Azure: `Patient` at 98–103 → via `ctx_map`, the same word	entity
9	`flag` (12)	Detectors azure, azure_ctx, rule (label has "NAME"), vlm all agree; no protection applies	flag: kind name, mask `SMITH` only, comma kept

Step	Function (cell)	What happens to `SMITH`	Data after the step
10	propagation (12)	`SMITH` becomes a seed; `C. SMITH` on the signature line is flagged too	more flags
11	grouping (12)	`SMITH,` + `CAROLINE` = one name entity (`M` is a protected caption token)	entity `SMITH, CAROLINE`
12	`build_surrogates` (13)	Split into prefix `""`, core `SMITH`, suffix `,`	parts
13	`_fake_alpha` (13)	5-letter names, 5 closest in width, HMAC-seeded pick → `DIANA`	`new_text = "DIANA,"`
14	`choose_styles` (14)	Line 7's words drawn in every font; Arial scores best	font arial.ttf, size 32
15	`erase_word` (14)	Ink pixels of `SMITH,` replaced with paper pixels from around it; the box lines stay	clean box
16	`draw_word` (14)	`DIANA,` drawn at the same left edge and baseline, same ink darkness, 0.45 px blur	action `replaced`
17	`save_verification_record` (batch)	Output re-read by Textract; the record stores `SMITH,` → `DIANA,` and what is now readable there	`_verify.json`

The same `SMITH` everywhere on the page, and on every other page with the same `DEID_SURROGATE_KEY`, becomes `DIANA`.

Why this works well

- **Positions and meaning come from different tools.** Textract gives exact positions; GPT-5.4 and Azure give meaning. Every answer is mapped back to Textract's word ids, so nothing is drawn in the wrong place.
- **Several detectors, one veto.** Any detector can flag a word, so one tool missing a value is not enough to leak it. Only the protection rules (captions, boilerplate, labels) can stop a flag, and they only protect printed form text.
- **Fakes that fit.** Same length, same case, same punctuation, closest width: the fake fits in the original's box without shrinking, and the layout of the form is unchanged.
- **Consistent fakes.** The HMAC seed makes the same value always give the same fake, and the single date shift keeps the time between dates true.
- **Looks like the original scan.** Font and size are measured on the page itself, form lines are never erased, background and ink colour are sampled, and a light blur matches the scan.
- **Checkable.** Five debug stages, a summary row per page and a verification record make every decision visible.

Things to know (found while going through the code)

- The solid-box fail-safe in cell 14 is switched off, so handwritten and tilted words are redrawn in a printed font.
- `M` and `F` in `FORM_TOKENS` keep a one-letter middle initial unchanged.
- The preview cell needs `import matplotlib.pyplot as plt`.
- `IMAGE_PATH`, `extracted_text_detailed.txt` and the batch folders are fixed Windows paths; change them for another machine.
- The verification record makes a second Textract call per page (about $0.0015).